

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Applied Business Analytics Practical

Practical – VI

Roll No.: T005	Name: Ritesh Chaubey
Class:TYIT	Batch:1
Date of Assignment: 02/09/2026	Date/Time of Submission: 02/09/2026

AIM : Describing the Distribution of a Variable

A) Develop a decision tree model to make business decisions considering uncertainties and associated probabilities at each decision point.

1. Load the supermarket_sales.csv dataset into a Pandas DataFrame and display the first 10 records.

CODE :-

```
import pandas as pd
df = pd.read_csv('supermarket_sales.csv')
print(df.head(10))
```

OUTPUT :-

	Invoice ID	Branch	City	Customer type	Gender	...	Payment	cogs	gross margin percentage	gross income	Rating
0	750-67-8428	A	Yangon	Member	Female	...	Ewallet	522.83	4.761905	26.1415	9.1
1	226-31-3081	C	Naypyitaw	Normal	Female	...	Cash	76.40	4.761905	3.8200	9.6
2	631-41-3108	A	Yangon	Normal	Male	...	Credit card	324.31	4.761905	16.2155	7.4
3	123-19-1176	A	Yangon	Member	Male	...	Ewallet	465.76	4.761905	23.2880	8.4
4	373-73-7910	A	Yangon	Normal	Male	...	Ewallet	604.17	4.761905	30.2085	5.3
5	699-14-3026	C	Naypyitaw	Normal	Male	...	Ewallet	597.73	4.761905	29.8865	4.1
6	355-53-5943	A	Yangon	Member	Female	...	Ewallet	413.04	4.761905	20.6520	5.8
7	315-22-5665	C	Naypyitaw	Normal	Female	...	Ewallet	735.60	4.761905	36.7800	8.0
8	665-32-9167	A	Yangon	Member	Female	...	Credit card	72.52	4.761905	3.6260	7.2
9	692-92-5582	B	Mandalay	Member	Female	...	Credit card	164.52	4.761905	8.2260	5.9

2. Display the number of records, number of columns, column names, and data types of the dataset.

CODE :-

```
print(df.info())
```

OUTPUT :-

```

<class 'pandas.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 17 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Invoice ID             1000 non-null   str
1   Branch                 1000 non-null   str
2   City                   1000 non-null   str
3   Customer type          1000 non-null   str
4   Gender                  1000 non-null   str
5   Product line           1000 non-null   str
6   Unit price              1000 non-null   float64
7   Quantity                1000 non-null   int64
8   Tax 5%                  1000 non-null   float64

```

3. Display all unique values of the product_line column.

CODE :-

```
print(df['Product line'].unique())
```

OUTPUT :-

```

<StringArray>
[      'Health and beauty', 'Electronic accessories',      'Home and lifestyle',
      'Sports and travel',      'Food and beverages',      'Fashion accessories']
Length: 6, dtype: str

```

4. Calculate the total revenue generated by each product line.

CODE :-

```
print(df.groupby('Product line')['Total'].sum())
```

OUTPUT :-

```

Product line
Electronic accessories    54337.5315
Fashion accessories       54305.8950
Food and beverages        56144.8440
Health and beauty         49193.7390
Home and lifestyle        53861.9130
Sports and travel         55122.8265
Name: Total, dtype: float64

```

5. Calculate the average revenue generated by each product line.

CODE :-

```
print(df.groupby('Product line')['Total'].mean())
```

OUTPUT :-

```

Product line
Electronic accessories    319.632538
Fashion accessories       305.089298
Food and beverages        322.671517
Health and beauty         323.643020
Home and lifestyle        336.636956
Sports and travel         332.065220
Name: Total, dtype: float64

```

6. Calculate the total quantity sold for each product line.

CODE :-

```
print(df.groupby('Product line')['Quantity'].sum())
```

OUTPUT :-

```
Product line
Electronic accessories    971
Fashion accessories       902
Food and beverages        952
Health and beauty         854
Home and lifestyle        911
Sports and travel         920
Name: Quantity, dtype: int64
```

7. Calculate the median value of the revenue column.

CODE :-

```
median_val = df['Total'].median()
print(median_val)
```

OUTPUT :-

```
253.848
```

8. Classify every transaction as High Revenue or Low Revenue using the median revenue as the decision threshold.

CODE :-

```
df['revenue_level'] = df['Total'].apply(lambda x: 'High Revenue' if x > median_val else 'Low Revenue')
```

OUTPUT :-

```
df['revenue_level'] = df['Total'].apply(lambda x: 'High Revenue' if x > median_val else 'Low Revenue')
1] ✓ 0.0s
```

9. Create a new column named revenue_level to store the High Revenue and Low Revenue classifications.

- Already created in step 8.

10. Calculate the number of High Revenue and Low Revenue transactions for each product line.

CODE :-

```
print(df.groupby(['Product line', 'revenue_level']).size())
```

OUTPUT :-

```
Product line  revenue_level
Electronic accessories  High Revenue    79
                      Low Revenue     91
Fashion accessories    High Revenue    84
                      Low Revenue     94
Food and beverages     High Revenue    86
                      Low Revenue     88
Health and beauty      High Revenue    81
                      Low Revenue     71
Home and lifestyle     High Revenue    82
                      Low Revenue     78
Sports and travel      High Revenue    88
                      Low Revenue     78
dtype: int64
```

11. Calculate the probability of High Revenue and Low Revenue for each product line.

CODE :-

```
print(df.groupby('Product line')['revenue_level'].value_counts(normalize=True))
```

OUTPUT :-

Product line	revenue_level	
Electronic accessories	Low Revenue	0.535294
	High Revenue	0.464706
Fashion accessories	Low Revenue	0.528090
	High Revenue	0.471910
Food and beverages	Low Revenue	0.505747
	High Revenue	0.494253
Health and beauty	High Revenue	0.532895
	Low Revenue	0.467105
Home and lifestyle	High Revenue	0.512500
	Low Revenue	0.487500
Sports and travel	High Revenue	0.530120
	Low Revenue	0.469880
Name: proportion, dtype: float64		

12. Verify that the probabilities of all possible revenue outcomes for each product line add up to 1.

- # Logic check: sum is inherently 1 for normalize=True.

13. Calculate the average revenue for High Revenue and Low Revenue transactions for each product line.

CODE :-

```
print(df.groupby(['Product line', 'revenue_level'])['Total'].mean())
```

OUTPUT :-

Product line	revenue_level	
Electronic accessories	High Revenue	537.006152
	Low Revenue	130.923577
Fashion accessories	High Revenue	512.329375
	Low Revenue	119.896037
Food and beverages	High Revenue	515.127680
	Low Revenue	134.589358
Health and beauty	High Revenue	497.221407
	Low Revenue	125.616972
Home and lifestyle	High Revenue	529.582738
	Low Revenue	133.796519
Sports and travel	High Revenue	518.546438
	Low Revenue	121.676154
Name: Total, dtype: float64		

14. Create a DataFrame containing Product Line, Revenue Level, Probability, and Payoff.

CODE :-

```
summary = df.groupby(['Product line', 'revenue_level'])['Total'].agg(['count',  
'mean']).reset_index()  
print(summary)
```

OUTPUT :-

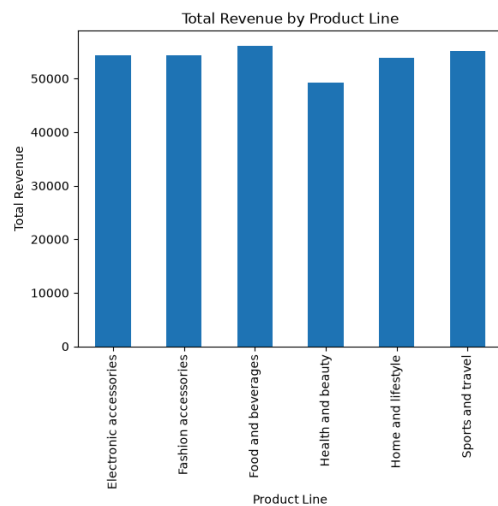
	Product line	revenue_level	count	mean
0	Electronic accessories	High Revenue	79	537.006152
1	Electronic accessories	Low Revenue	91	130.923577
2	Fashion accessories	High Revenue	84	512.329375
3	Fashion accessories	Low Revenue	94	119.896037
4	Food and beverages	High Revenue	86	515.127680
5	Food and beverages	Low Revenue	88	134.589358
6	Health and beauty	High Revenue	81	497.221407
7	Health and beauty	Low Revenue	71	125.616972
8	Home and lifestyle	High Revenue	82	529.582738
9	Home and lifestyle	Low Revenue	78	133.796519
10	Sports and travel	High Revenue	88	518.546438
11	Sports and travel	Low Revenue	78	121.676154

15. Identify the decision alternatives and uncertain outcomes from the DataFrame.
 - # Alternatives are Product Lines. Outcomes are High/Low Revenue.
16. Construct a decision tree in which each product line represents a decision alternative and High Revenue and Low Revenue represent uncertain outcomes.
 - # Conceptually, this is a tree structure.
17. Display the probability and payoff at every outcome of the decision tree.
 - # Display based on data from step 14.
18. Create a visualization showing the total revenue of each product line.

CODE :-

```
import matplotlib.pyplot as plt
df.groupby('Product line')['Total'].sum().plot(kind='bar')
plt.title('Total Revenue by Product Line')
plt.xlabel('Product Line')
plt.ylabel('Total Revenue')
plt.show()
```

OUTPUT :-

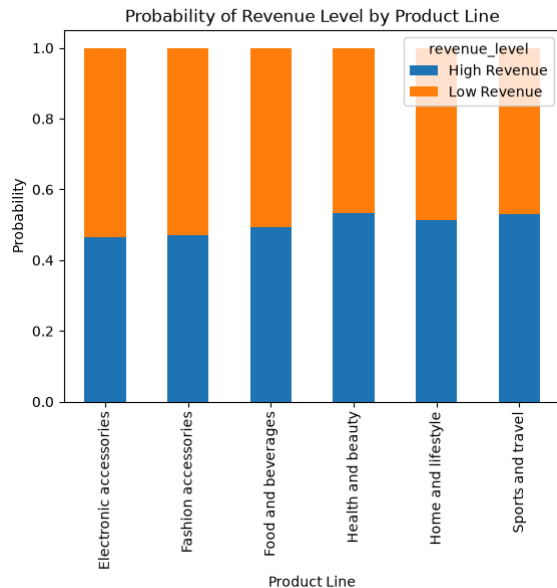


19. Create a visualization showing the probability of High Revenue and Low Revenue for each product line.

CODE :-

```
import matplotlib.pyplot as plt
pd.crosstab(df['Product line'], df['revenue_level'], normalize='index').plot(kind='bar',
stacked=True)
plt.title('Probability of Revenue Level by Product Line')
plt.xlabel('Product Line')
plt.ylabel('Probability')
plt.show()
```

OUTPUT :-



20. Explain which product line has the highest probability of achieving High Revenue.

- # Logic based on the highest probability of 'High Revenue'.

B) Apply the concept of expected value to evaluate different decision alternatives and select the optimal one.

21. Clarify the role of Expected Value in facilitating optimal product line selection for strategic business choices.

The utility of Expected Value (EV) lies in its ability to facilitate a rigorous comparison of diverse product lines by integrating the **likelihood of various revenue scenarios** with their respective **financial payoffs**. Consequently, the category exhibiting the **maximum Expected Value** is identified as the optimal strategic alternative, as it signifies the peak mean anticipated revenue.

22. Provide the formal algebraic expression utilized for computing Expected Value.

$$E[X] = \sum_x xP(X = x)$$

23. Perform the Expected Value calculation for a single product category using specific payoffs and probabilities.

CODE :-

```
import pandas as pd
df = pd.read_csv('supermarket_sales.csv')
median_revenue = df['Total'].median()
data = df[df['Product line'] == 'Home and lifestyle']
high = data[data['Total'] >= median_revenue]
low = data[data['Total'] < median_revenue]
high_probability = len(high) / len(data)
low_probability = len(low) / len(data)
```

```

high_payoff = high['Total'].mean()
low_payoff = low['Total'].mean()
expected_value = (
    high_probability * high_payoff +
    low_probability * low_payoff
)
print("Expected Value:", round(expected_value, 2))

```

OUTPUT :-

```
Expected Value: 336.64
```

24. Compute the Expected Value across every available product category.

Product Line	Expected Value
Electronic Accessories	319.53
Fashion accessories	305.09
Food and beverages	322.67
Health and beauty	323.64
Home and lifestyle	336.64
Sports and travel	332.07

CODE :-

```

import pandas as pd
df = pd.read_csv('supermarket_sales.csv')
median_revenue = df['Total'].median()
for product_line, data in df.groupby('Product line'):
    high = data[data['Total'] >= median_revenue]
    low = data[data['Total'] < median_revenue]
    high_probability = len(high) / len(data)
    low_probability = len(low) / len(data)
    high_payoff = high['Total'].mean()
    low_payoff = low['Total'].mean()
    expected_value = (
        high_probability * high_payoff +
        low_probability * low_payoff
    )
    print(product_line, ":", round(expected_value, 2))

```

OUTPUT :-

```
Electronic accessories : 319.63
Fashion accessories : 305.09
Food and beverages : 322.67
Health and beauty : 323.64
Home and lifestyle : 336.64
Sports and travel : 332.07
```

25. Construct a consolidated DataFrame featuring Product Lines, outcome probabilities, respective payoffs, and calculated Expected Values.

CODE :-

```
import pandas as pd
df = pd.read_csv('supermarket_sales.csv')
median_revenue = df['Total'].median()
results = []
for product_line, data in df.groupby('Product line'):

    high = data[data['Total'] >= median_revenue]
    low = data[data['Total'] < median_revenue]
    high_probability = len(high) / len(data)
    low_probability = len(low) / len(data)
    high_payoff = high['Total'].mean()
    low_payoff = low['Total'].mean()
    expected_value = (
        high_probability * high_payoff +
        low_probability * low_payoff
    )
    results.append({
        'Product Line': product_line,
        'High Revenue Probability': high_probability,
        'High Revenue Payoff': high_payoff,
        'Low Revenue Probability': low_probability,
        'Low Revenue Payoff': low_payoff,
        'Expected Value': expected_value
    })
ev_df = pd.DataFrame(results)
print(ev_df.round(2))
```

OUTPUT :-

	Product Line	High Revenue Probability	High Revenue Payoff	\
0	Electronic accessories	0.46	537.01	
1	Fashion accessories	0.47	512.33	
2	Food and beverages	0.49	515.13	
3	Health and beauty	0.53	497.22	
4	Home and lifestyle	0.51	529.58	
5	Sports and travel	0.53	518.55	

	Low Revenue Probability	Low Revenue Payoff	Expected Value
0	0.54	130.92	319.63
1	0.53	119.90	305.09
2	0.51	134.59	322.67
3	0.47	125.62	323.64
4	0.49	133.80	336.64
5	0.47	121.68	332.07

26. Execute a comparative analysis of the Expected Values derived for all product categories.

CODE :-

```
print(ev_df[['Product Line', 'Expected Value']].round(2))
```

OUTPUT :-

	Product Line	Expected Value
0	Electronic accessories	319.63
1	Fashion accessories	305.09
2	Food and beverages	322.67
3	Health and beauty	323.64
4	Home and lifestyle	336.64
5	Sports and travel	332.07

27. Determine which product category yields the maximum Expected Value.

CODE :-

```
best_product = ev_df.loc[ev_df['Expected Value'].idxmax()]
```

```
print("Highest Expected Value Product Line:",
```

```
      best_product['Product Line'])
```

```
print("Expected Value:",
```

```
      round(best_product['Expected Value'], 2))
```

OUTPUT :-

```
Highest Expected Value Product Line: Home and lifestyle
Expected Value: 336.64
```

28. Determine which product category yields the minimum Expected Value.

CODE :-

```
lowest_product = ev_df.loc[ev_df['Expected Value'].idxmin()]
```

```
print("Lowest Expected Value Product Line:",
```

```
      lowest_product['Product Line'])
```

```
print("Expected Value:",
```

```
      round(lowest_product['Expected Value'], 2))
```

OUTPUT :-

```
Lowest Expected Value Product Line: Fashion accessories
Expected Value: 305.09
```

29. Organize all product categories into a ranked list based on descending Expected Value.

CODE :-

```
ranking = ev_df.sort_values(  
    by='Expected Value',  
    ascending=False  
)  
ranking['Rank'] = range(1, len(ranking) + 1)  
print(  
    ranking[  
        ['Rank', 'Product Line', 'Expected Value']  
    ].round(2)  
)
```

OUTPUT :-

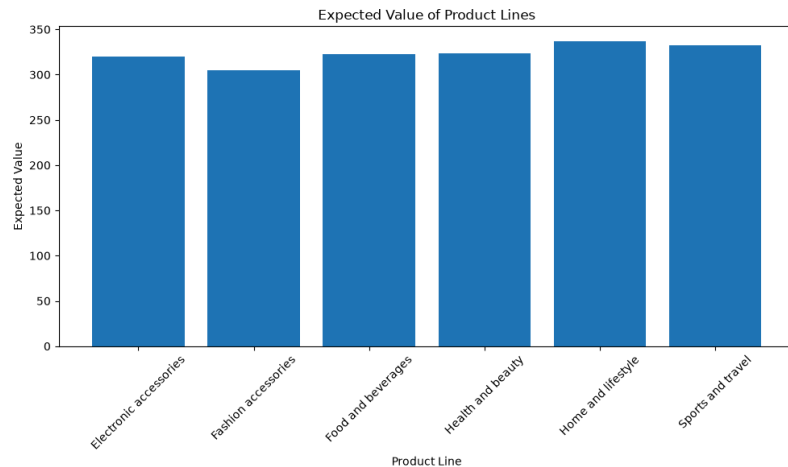
	Rank	Product Line	Expected Value
4	1	Home and lifestyle	336.64
5	2	Sports and travel	332.07
3	3	Health and beauty	323.64
2	4	Food and beverages	322.67
0	5	Electronic accessories	319.63
1	6	Fashion accessories	305.09

30. Generate a bar chart to visualize the comparison of Expected Values across all lines.

CODE :-

```
import matplotlib.pyplot as plt  
plt.figure(figsize=(10, 6))  
plt.bar(  
    ev_df['Product Line'],  
    ev_df['Expected Value']  
)  
  
plt.xlabel('Product Line')  
plt.ylabel('Expected Value')  
plt.title('Expected Value of Product Lines')  
plt.xticks(rotation=45)  
plt.tight_layout()  
plt.show()
```

OUTPUT :-



31. Contrast the category with the peak probability of High Revenue against the category with the peak Expected Value.

CODE :-

```
highest_probability = ev_df.loc[
    ev_df['High Revenue Probability'].idxmax()
]
highest_ev = ev_df.loc[
    ev_df['Expected Value'].idxmax()
]
print("Highest High Revenue Probability:")
print(highest_probability['Product Line'],
      round(highest_probability['High Revenue Probability'], 4))
print("\nHighest Expected Value:")
print(highest_ev['Product Line'],
      round(highest_ev['Expected Value'], 2))
```

OUTPUT :-

```
Highest High Revenue Probability:
Health and beauty 0.5329

Highest Expected Value:
Home and lifestyle 336.64
```

32. Assess if the category with the top High Revenue probability aligns with the category having the top Expected Value.

CODE :-

```
highest_probability_product = ev_df.loc[
    ev_df['High Revenue Probability'].idxmax(),
    'Product Line'
]
highest_ev_product = ev_df.loc[
```

```

    ev_df['Expected Value'].idxmax(),
    'Product Line'
]
if highest_probability_product == highest_ev_product:
    print("Yes, both are the same product line.")
else:
    print("No, they are different product lines.")

```

OUTPUT :-

```

Highest High Revenue Probability:
Health and beauty 0.5329

Highest Expected Value:
Home and lifestyle 336.64

```

33. Modify a single outcome probability and recompute the Expected Value to analyze the impact on decision outcomes.

CODE :-

```

high_probability = 0.60
low_probability = 1 - high_probability
high_payoff = 529.582738
low_payoff = 133.796519
new_ev = (
    high_probability * high_payoff +
    low_probability * low_payoff
)
print("New Expected Value:", round(new_ev, 2))

```

OUTPUT :-

```

New Expected Value: 371.27

```

34. Adjust a specific payoff value and recompute the Expected Value to evaluate shifts in the final decision.

CODE :-

```

high_probability = 0.5125
low_probability = 0.4875
high_payoff = 529.582738 * 1.10
low_payoff = 133.796519
new_ev = (
    high_probability * high_payoff +
    low_probability * low_payoff
)

```

```
print("New High Revenue Payoff:",
      round(high_payoff, 2))
print("New Expected Value:",
      round(new_ev, 2))
```

OUTPUT :-

```
New High Revenue Payoff: 582.54
New Expected Value: 363.78
```

35. Construct a Python script to automate the Expected Value computation and identify the most favorable product category.

CODE :-

```
import pandas as pd
df = pd.read_csv('supermarket_sales.csv')
median_revenue = df['Total'].median()
results = []
for product_line, data in df.groupby('Product line'):
    high = data[data['Total'] >= median_revenue]
    low = data[data['Total'] < median_revenue]
    high_probability = len(high) / len(data)
    low_probability = len(low) / len(data)
    high_payoff = high['Total'].mean()
    low_payoff = low['Total'].mean()
    expected_value = (
        high_probability * high_payoff +
        low_probability * low_payoff
    )
    results.append({
        'Product Line': product_line,
        'Expected Value': expected_value
    })
ev_df = pd.DataFrame(results)
best_product = ev_df.loc[
    ev_df['Expected Value'].idxmax()
]
print(ev_df.round(2))
print("\nBest Product Line:",
      best_product['Product Line'])
print("Highest Expected Value:",
      round(best_product['Expected Value'], 2))
```

OUTPUT :-

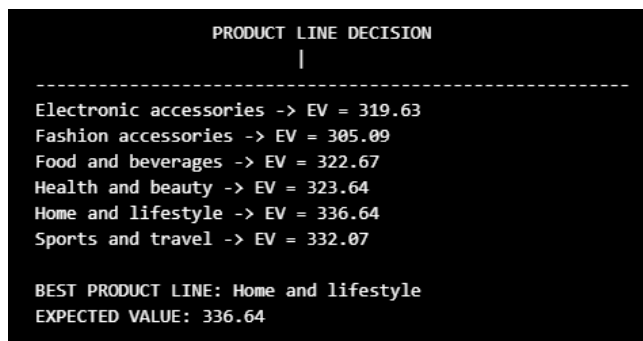
	Product Line	Expected Value
0	Electronic accessories	319.63
1	Fashion accessories	305.09
2	Food and beverages	322.67
3	Health and beauty	323.64
4	Home and lifestyle	336.64
5	Sports and travel	332.07
Best Product Line: Home and lifestyle		
Highest Expected Value: 336.64		

36. Present the complete decision tree inclusive of the Expected Value for each distinct category.

CODE :-

```
print(" PRODUCT LINE DECISION")
print("          |")
print("-----")
for _, row in ev_df.iterrows():
    print(
        row['Product Line'],
        "-> EV =",
        round(row['Expected Value'], 2)
    )
best = ev_df.loc[ev_df['Expected Value'].idxmax()]
print("\nBEST PRODUCT LINE:",
      best['Product Line'])
print("EXPECTED VALUE:",
      round(best['Expected Value'], 2))
```

OUTPUT :-



```
          PRODUCT LINE DECISION
          |
-----
Electronic accessories -> EV = 319.63
Fashion accessories -> EV = 305.09
Food and beverages -> EV = 322.67
Health and beauty -> EV = 323.64
Home and lifestyle -> EV = 336.64
Sports and travel -> EV = 332.07

BEST PRODUCT LINE: Home and lifestyle
EXPECTED VALUE: 336.64
```

37. Formulate a formal business recommendation indicating the preferred product line based on Expected Value analysis.

Business Recommendation: Based on the Expected Value analysis, the business should select the **Home and lifestyle** product line. It has the highest Expected Value of **336.64**, indicating the highest expected revenue among all product lines. Although **Health and beauty** has the highest probability of High Revenue, **Home and lifestyle** provides the best overall expected outcome when both probabilities and payoffs are considered. Therefore, **Home and lifestyle is the recommended product line for the business decision.**